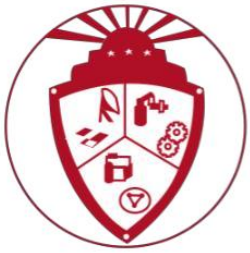
UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



FACULTAD DE INGENIERÍA EN
SISTEMAS, ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

Título: Reto práctico integrador SJA-UTA: árbol general y árbol binario de búsqueda

Estudiantes:

Cunalata Mendoza Damian Alexander

Tacuri Santillan Mónica Sara

Chalco Tasna Kenneth Mateo

Silva Camuendo Luis Alexander

Tisalema Guashco Darwin Joel

Camacho Monta Josue Jampier

Modalidad:

Grupal – Reto práctico integrador APE 3 (Grupo 2)

Ciclo Académico y Paralelo:

Tercero “B”

Módulo y Docente:

Estructura de Datos – Ing. José Caiza.

JULIO 2026 - DICIEMBRE 2026

Índice

1. Introducción 3
2. Objetivos 3
2.1. Objetivo general 3
2.2. Objetivos específicos 3
3. Repositorio del proyecto 3
4. Descripción del reto 4
5. Organización del trabajo en equipo 5
5.1. Reparto por clases 5
5.2. Diseño: delegación desde ArbolOrganizacional 5
5.3. Flujo de trabajo con Git 5
6. Desarrollo de las misiones 7
6.1. Misión 1: jerarquía y ruta institucional (Damian Cunalata) 7
6.2. Misión 2: protección de unidades y transformación binaria (Mónica Tacuri) 9
6.3. Misión 3: validación de datos y traslado entre paralelos (Kenneth Chalco) 10
6.4. Misión 4: modificación segura y casos de eliminación del ABB (Luis Silva) 12
6.5. Misión 5: recorridos y estadísticas comparadas (Darwin Tisalema) 14
6.6. Misión 6: integración y reinicio (Damian Cunalata) 15
7. Compilación y verificaciones obligatorias 17
8. Evidencias de commits por integrante 17
8.1. Damian Cunalata 18
8.2. Mónica Tacuri 18
8.3. Kenneth Chalco 19
8.4. Luis Silva 20
8.5. Darwin Tisalema 20
8.6. Josue Jampier Camacho 21
9. Decisiones de implementación 22
10. Conclusiones 22
11. Recomendaciones 22
12. Anexos 22

1. Introducción

El Sistema Jerárquico Académico UTA (SJA-UTA) es una aplicación de consola en Java que representa la organización de la Universidad Técnica de Ambato mediante un **árbol general** (universidad, facultades, carreras, niveles y paralelos) y administra los estudiantes de cada paralelo mediante un **árbol binario de búsqueda (ABB)** ordenado por matrícula.

Este informe documenta el reto práctico integrador de la APE 3 desarrollado por el Grupo 2. Se describen las seis misiones del reto, la forma en que se repartió el trabajo (una persona responsable por cada clase), los diagramas de cada solución, las capturas de una ejecución real del programa y los commits más relevantes de cada integrante en el repositorio de GitHub.

2. Objetivos

2.1. Objetivo general

Completar y comprobar el SJA-UTA a partir del código base entregado, integrando el árbol general y el ABB, y demostrar que la estructura académica, los registros estudiantiles y las reglas de negocio se mantienen consistentes después de modificaciones, eliminaciones y traslados.

2.2. Objetivos específicos

1. Implementar de forma recursiva la ruta jerárquica de una unidad académica (Misión 1).
2. Proteger la eliminación de unidades y construir la representación binaria hijo izquierdo / hermano derecho sin alterar el árbol original (Misión 2).
3. Validar los datos del estudiante y trasladarlo entre paralelos sin perder información (Misión 3).
4. Comprobar la modificación segura y los cuatro casos de eliminación del ABB: hoja, un hijo, dos hijos y raíz (Misión 4).
5. Calcular estadísticas de los paralelos y contar recursivamente los estudiantes aprobados (Misión 5).
6. Integrar todo en la opción 8 del menú y garantizar que el reinicio recupere los datos originales (Misión 6).

3. Repositorio del proyecto

El código fuente, la documentación y las evidencias se encuentran en el repositorio público de GitHub: https://github.com/Damienq7w/Estructura_Datos_Reto_Grupo4

El repositorio terminado tiene 6 colaboradores, 30 commits y 7 pull requests fusionados en la rama `main`. El lenguaje es 100 % Java. Para compilar y ejecutar el proyecto desde la raíz del repositorio (JDK 21):

```
javac -encoding UTF-8 -d out (Get-ChildItem -Recurse src -Filter
*.java).FullName
java -cp out app.Main             # menú interactivo
java -cp out app.RetoIntegrador   # ejecuta el reto completo
```

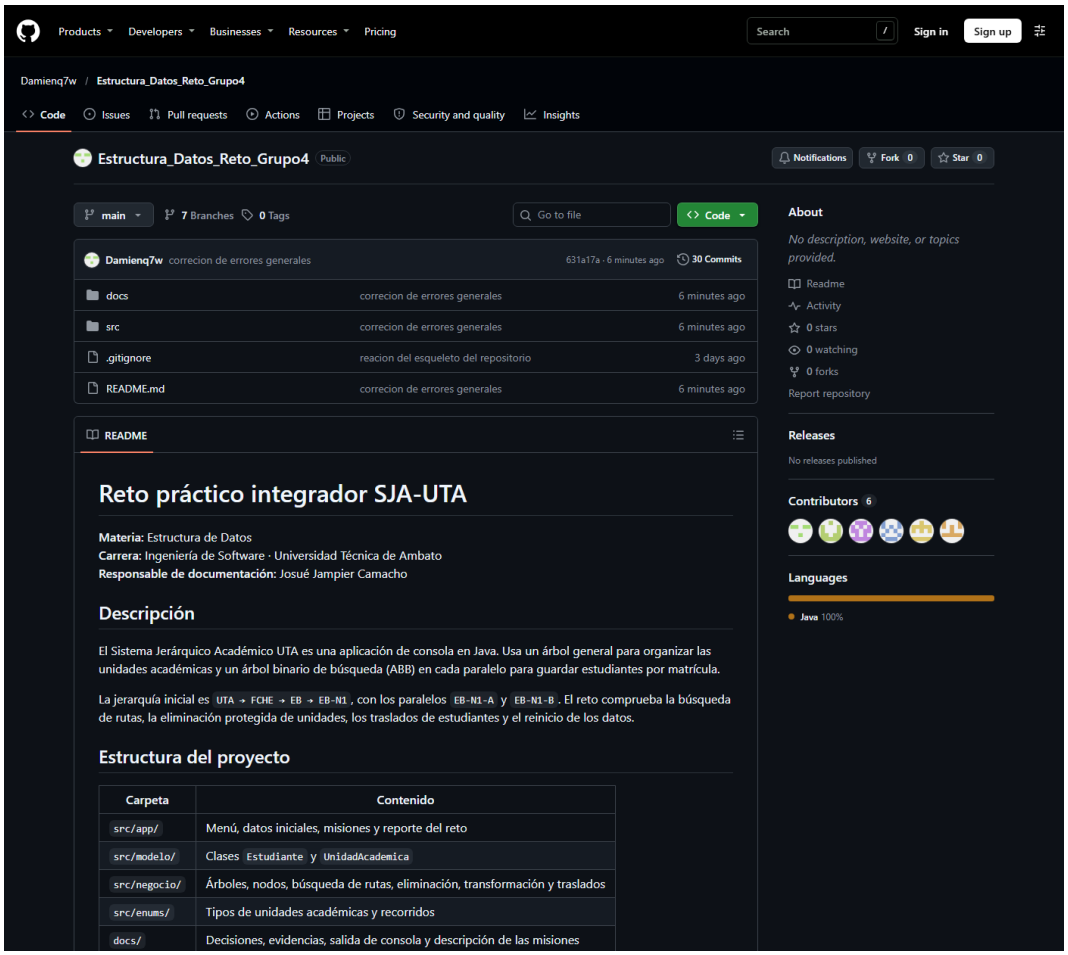


Figura 1. *Repositorio terminado en GitHub: carpetas, README, 30 commits y 6 colaboradores.*

4. Descripción del reto

Se partió del proyecto SJA-UTA entregado, conservando los paquetes `enums`, `modelo`, `negocio` y `app` y las firmas existentes. La tabla 1 resume las condiciones iniciales.

Tabla 1. *Condiciones iniciales del reto.*

Elemento	Valor inicial
Jerarquía	UTA → FCHE → EB → EB-N1, con los paralelos EB-N1-A y EB-N1-B
Estudiantes de EB-N1-A	Matrículas 30, 15, 45, 10, 20, 40, 50, 35 y 55 (en ese orden), todas con promedio 8.00
Estudiantes de EB-N1-B	Ninguno (paralelo vacío)
Altura	Se mide en aristas: un nodo = 0 y un árbol vacío = −1
Restricciones	Nodos propios, sin TreeMap ni TreeSet; operaciones recursivas y validaciones en la capa de negocio

5. Organización del trabajo en equipo

5.1. Reparto por clases

Cada clase tiene **un solo responsable**. Así ningún integrante modificó el archivo de otro y cada commit muestra con claridad el aporte de su autor.

Tabla 2. *Responsable de cada clase.*

Integrante	Misión	Clases a su cargo
Damian Cunalata	1 y 6	ArbolOrganizacional, RutaJerarquica, Contexto, Main, RetoIntegrador, RetoMision1
Mónica Tacuri	2	NodoUnidad, NodoBinario, EliminadorUnidades, TransformadorBinario, RetoMision2
Kenneth Chalco	3	Estudiante, UnidadAcademica, TipoUnidad, TrasladoEstudiantes, RetoMision3
Luis Silva	4	NodoEstudiante, Reporte, RetoMision4
Darwin Tisalema	5	TipoRecorrido, ArbolEstudiantes, RetoMision5
Josue Jampier Camacho	Documentación	README.md, docs/decisiones.md, docs/evidencias.md, docs/salida_reto.txt

5.2. Diseño: delegación desde ArbolOrganizacional

El enunciado exige que la ruta, la eliminación, la transformación binaria y el traslado estén en `ArbolOrganizacional`. Para que esa clase tuviera un único responsable, conserva las firmas pedidas y delega cada operación en una clase auxiliar de quien la programa (figura 2).

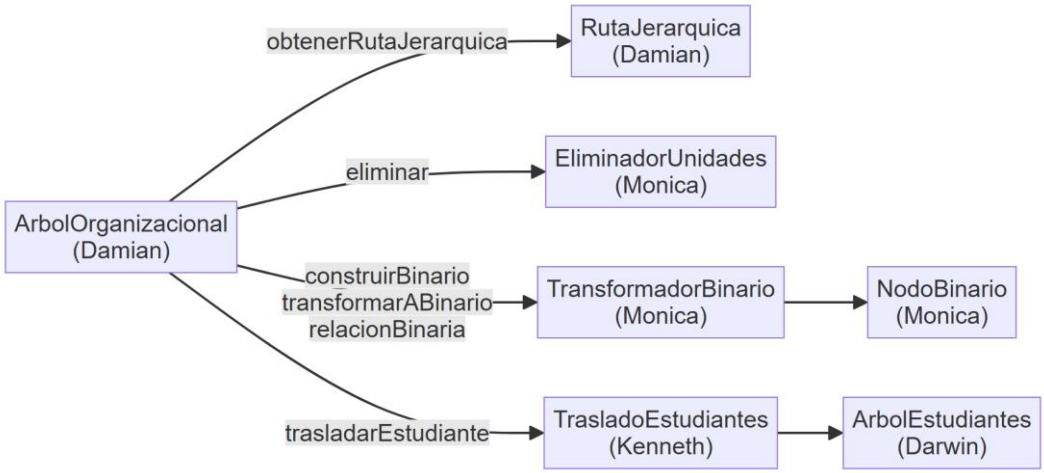


Figura 2. *ArbolOrganizacional delega cada operación del reto en la clase de su responsable.*

5.3. Flujo de trabajo con Git

Cada integrante trabajó en su propia rama y entregó su aporte mediante un pull request hacia `main`, que Damian revisó e integró.

Tabla 3. *Ramas y pull requests de cada integrante.*

Integrante	Rama	Pull request
Damian Cunalata	Damian_Cunalata	#1
Darwin Tisalema	Rama-Joel	#2
Kenneth Chalco	Mateo-Chalco	#3
Luis Silva	Luis-Silva	#4
Josue Jampier Camacho	rama-Josue	#5 y #6
Mónica Tacuri	Monica_Tacuri	#7

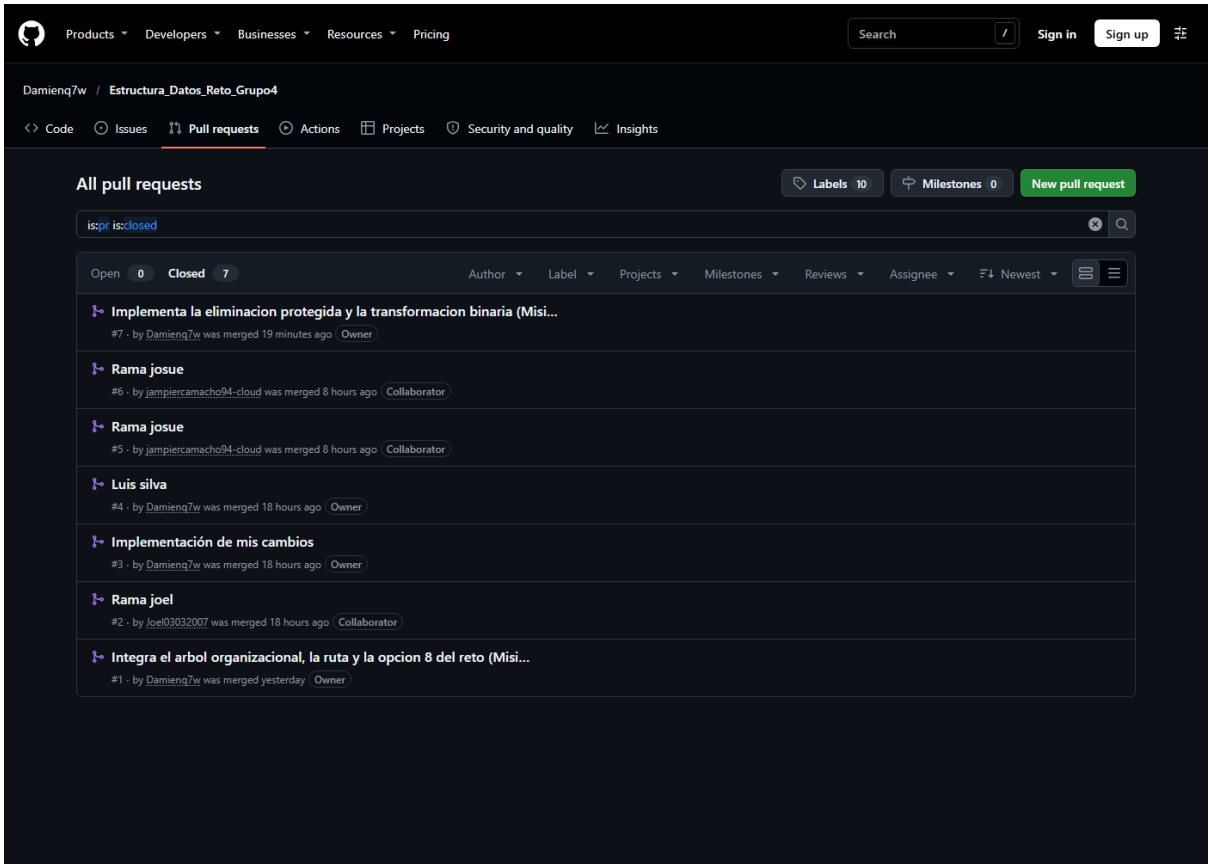


Figura 3. *Los 7 pull requests fusionados en main.*

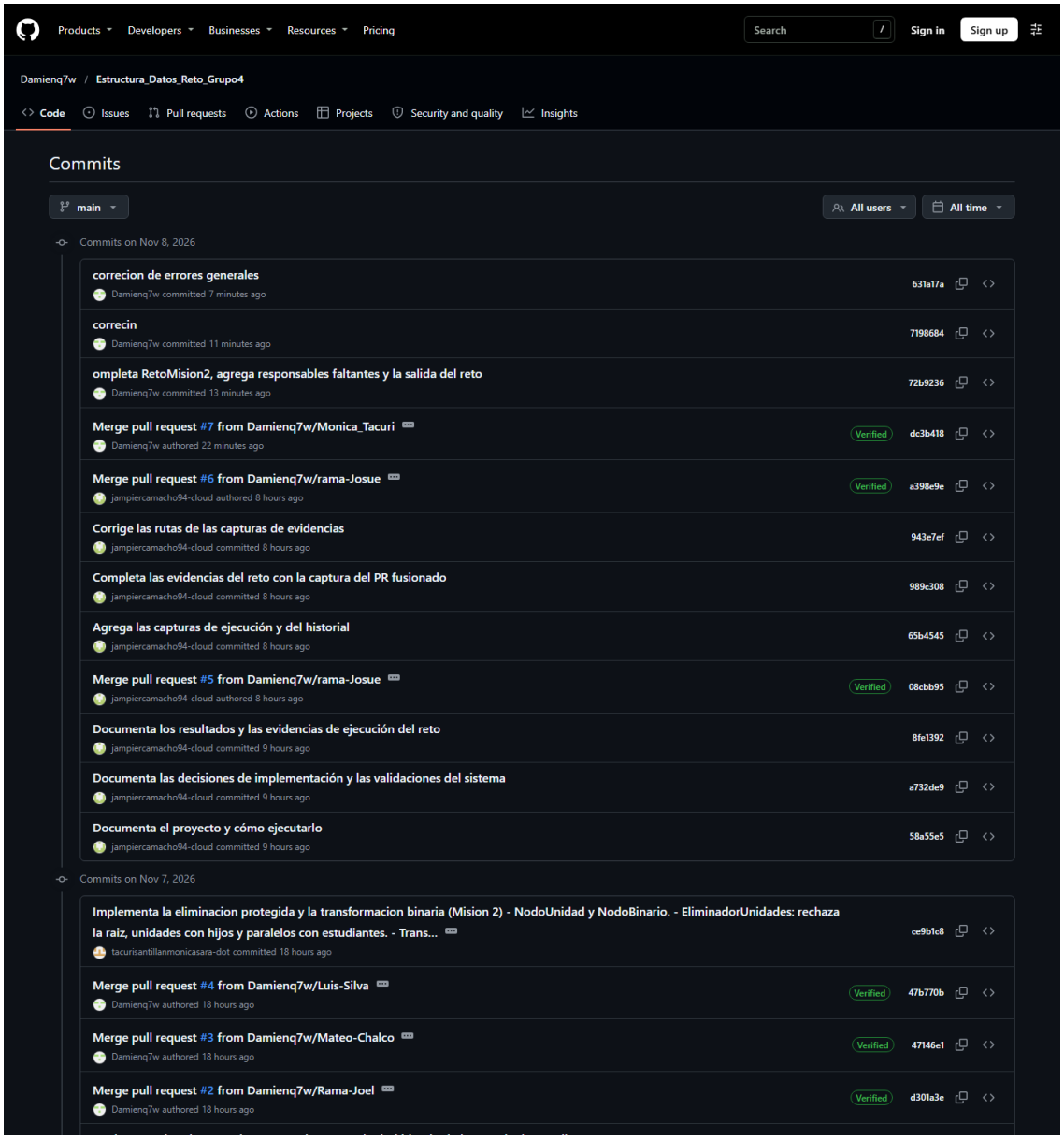


Figura 4. *Historial de commits de la rama main en GitHub.*

6. Desarrollo de las misiones

6.1. Misión 1: jerarquía y ruta institucional (Damian Cunalata)

Se registraron bajo FCHE la carrera SW, el nivel SW-N3 y el paralelo SW-N3-A, respetando la regla Universidad → Facultad → Carrera → Nivel → Paralelo. También se comprobó que se rechacen un código repetido, un salto de jerarquía y un padre inexistente.

`obtenerRutaJerarquica(String codigo)` es recursivo: si el nodo actual es la unidad buscada, devuelve su código (caso base); si no, busca en cada hijo y, cuando un hijo devuelve una ruta, le antepone el código actual y « -> ». Si no la encuentra, devuelve «No existe». Su complejidad es O(N).

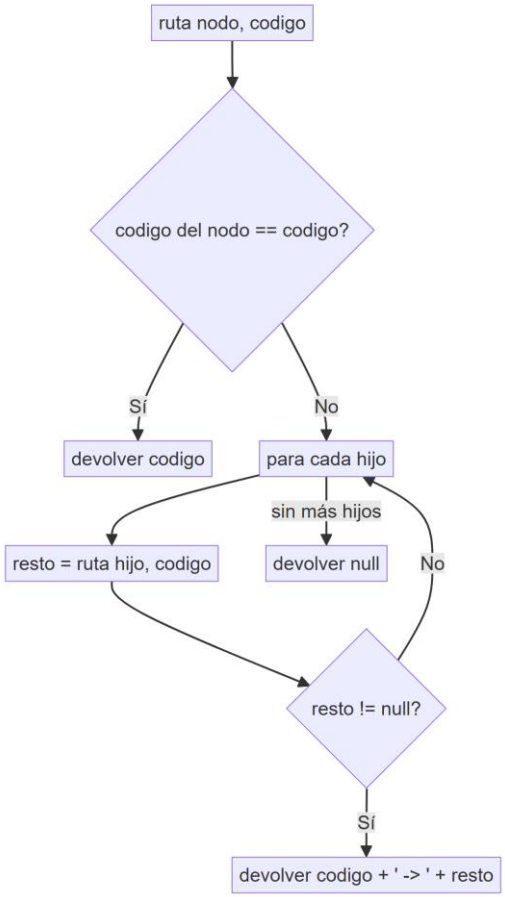


Figura 5. *Diagrama de flujo de la ruta jerárquica recursiva.*

Tabla 4. *Resultados de la Misión 1.*

Consulta	**Resultado obtenido**
Nivel de SW / SW-N3 / SW-N3-A	2 / 3 / 4
Altura del árbol	4 aristas
Cantidad de hojas	3 (EB-N1-A, EB-N1-B y SW-N3-A)
Ruta de EB-N1-A	UTA -> FCHE -> EB -> EB-N1 -> EB-N1-A
Ruta de XYZ	No existe

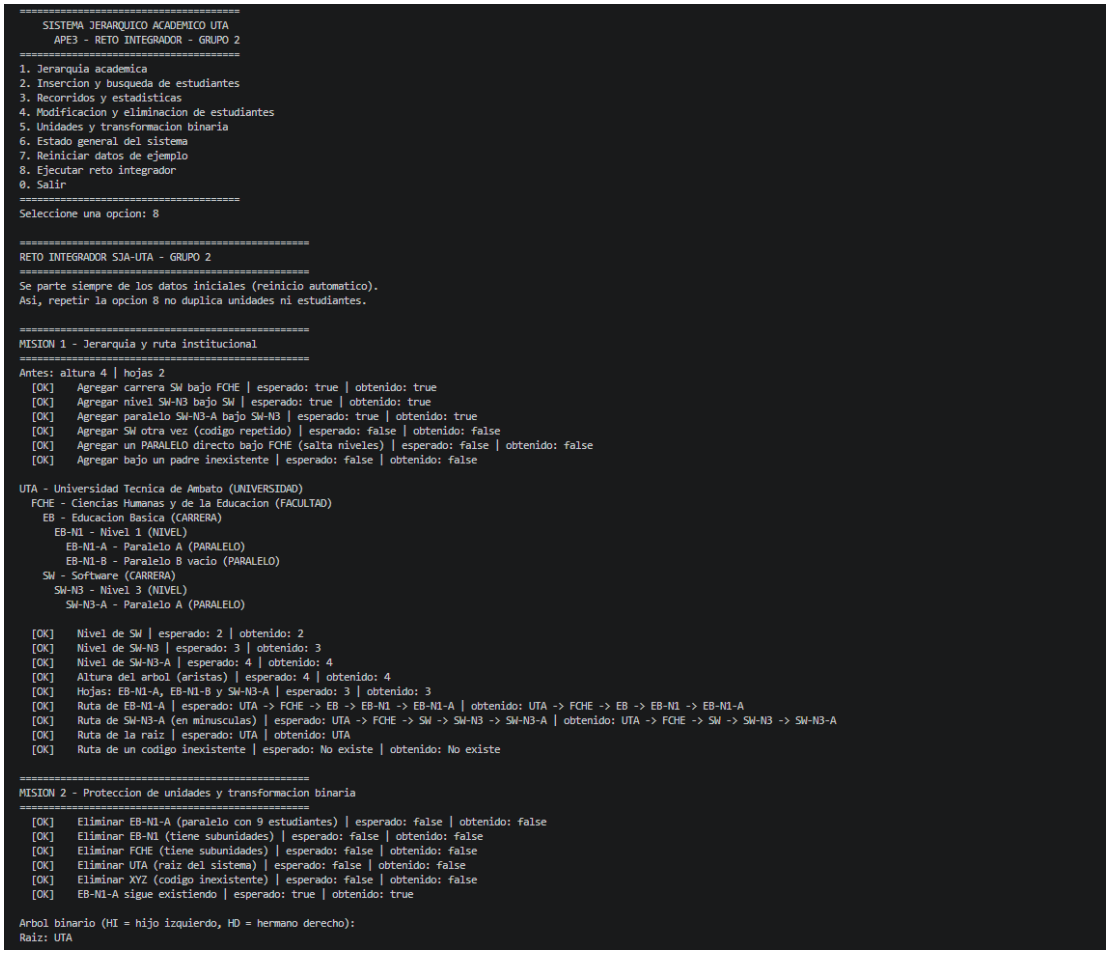


Figura 6. *Opción 8: inicio del reto, Misión 1 completa y comienzo de la Misión 2.*

6.2. Misión 2: protección de unidades y transformación binaria (Mónica Tacuri)

`EliminadorUnidades` solo permite eliminar una **hoja vacía**: rechaza la raíz UTA, las unidades con subunidades, los paralelos con estudiantes y los códigos inexistentes. La regla está en la capa de negocio, no solo en el menú.

Tabla 5. *Protección de unidades.*

Unidad	Condición	Resultado de eliminar
EB-N1-A	Paralelo con estudiantes	false
EB-N1 y FCHE	Unidades con subunidades	false
UTA	Raíz de la organización	false
XYZ	Código inexistente	false
SW-N3-A	Hoja vacía	true

`TransformadorBinario` construye un árbol binario **nuevo**: el primer hijo de cada unidad pasa a ser su hijo izquierdo (HI) y el siguiente hermano, su hermano derecho (HD). Como los nodos binarios son nuevos, el árbol general no se modifica; el reto lo comprueba comparando la jerarquía antes y después.

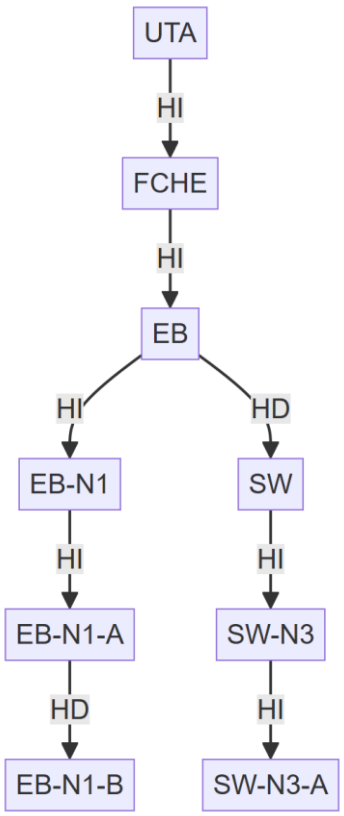


Figura 7. *Representación hijo izquierdo / hermano derecho: EB → HD SW y EB-N1-A → HD EB-N1-B.*

```
MISION 1 - Jerarquia y ruta institucional
  [OK]    Ruta de EB-N1-A | esperado: UTA -> FCHE -> EB -> EB-N1 -> EB-N1-A | obtenido: UTA -> FCHE -> EB -> EB-N1 -> EB-N1-A
MISION 2 - Proteccion de unidades y transformacion binaria
Arbol binario (HI = hijo izquierdo, HD = hermano derecho):
Raiz: UTA
  HI de UTA: FCHE
    HI de FCHE: EB
      HI de EB: EB-N1
        HI de EB-N1: EB-N1-A
          HD de EB-N1-A: EB-N1-B
      HD de EB: SW
        HI de SW: SW-N3
          HI de SW-N3: SW-N3-A
```

Figura 8. *Ejecución de las Misiones 1 y 2: ruta institucional y árbol binario.*

6.3. Misión 3: validación de datos y traslado entre paralelos (Kenneth Chalco)

`Estudiante` rechaza matrículas no positivas y promedios fuera de 0 a 10. Para `NaN` e `Infinity` se usa `Double.isFinite`, porque cualquier comparación con `NaN` da falso y esos valores pasarían una validación de rango simple.

`TrasladoEstudiantes` acepta el traslado solo si origen y destino existen, son distintos, son de tipo PARALELO, el estudiante está en el origen y su matrícula no está en el destino. Para no perder información, **primero inserta una copia en el destino** y luego elimina del origen; si esa eliminación falla, quita la copia del destino y el estado vuelve a ser el anterior.

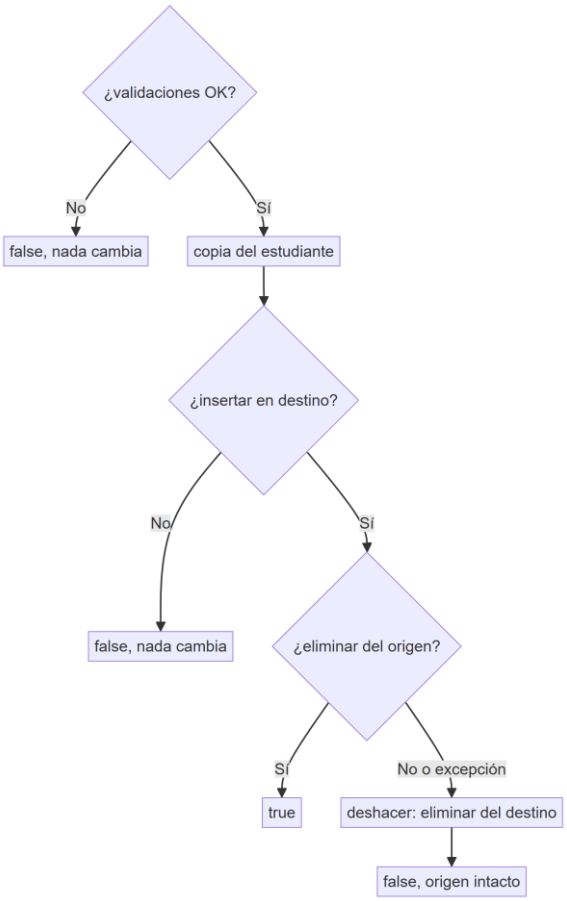


Figura 9. *Traslado seguro con recuperación del estado anterior.*

```
Raiz: UTA
  HI de UTA: FCHE
    HI de FCHE: EB
      HI de EB: EB-N1
        HI de EB-N1: EB-N1-A
          HD de EB-N1-A: EB-N1-B
      HD de EB: SW
        HI de SW: SW-N3
          HI de SW-N3: SW-N3-A

  [OK]    Relacion de FCHE | esperado: FCHE: HI = EB | HD = null | obtenido: FCHE: HI = EB | HD = null
  [OK]    EB tiene a SW como hermano derecho | esperado: EB: HI = EB-N1 | HD = SW | obtenido: EB: HI = EB-N1 | HD = SW
  [OK]    EB-N1-A tiene a EB-N1-B como hermano derecho | esperado: EB-N1-A: HI = null | HD = EB-N1-B | obtenido: EB-N1-A: HI = null | HD = EB-N1-B
  [OK]    SW es el ultimo hermano | esperado: SW: HI = SW-N3 | HD = null | obtenido: SW: HI = SW-N3 | HD = null
  [OK]    La transformacion no altera el arbol general
  [OK]    Eliminar SW-N3-A (hoja vacia) | esperado: true | obtenido: true
  [OK]    SW-N3-A ya no existe | esperado: null | obtenido: null
  [OK]    SW-N3 queda como hoja (grado 0) | esperado: 0 | obtenido: 0
  [OK]    Eliminar SW-N3-A por segunda vez | esperado: false | obtenido: false

==================================================
MISION 3 - Validacion de datos y traslado entre paralelos
==================================================
Validaciones de Estudiante:
  [OK]    Matricula 0 rechazada
  [OK]    Matricula negativa rechazada
  [OK]    Promedio -0.5 rechazado
  [OK]    Promedio 10.5 rechazado
  [OK]    Promedio NaN rechazado
  [OK]    Promedio Infinity rechazado
  [OK]    Promedio -Infinity rechazado
  [OK]    Promedio 0 aceptado (limite)
  [OK]    Promedio 10 aceptado (limite)
  [OK]    setPromedio(NaN) rechazado y conserva 7.5

Traslados que deben rechazarse (no cambian ningun arbol):
  [OK]    Origen inexistente | esperado: false | obtenido: false
  [OK]    Destino inexistente | esperado: false | obtenido: false
  [OK]    Destino que no es PARALELO (EB-N1) | esperado: false | obtenido: false
  [OK]    Origen que no es PARALELO (FCHE) | esperado: false | obtenido: false
  [OK]    Origen igual al destino | esperado: false | obtenido: false
  [OK]    Matricula 99 no existe en el origen | esperado: false | obtenido: false
  [OK]    EB-N1-A conserva sus 9 estudiantes | esperado: 10 15 20 30 35 40 45 50 55 | obtenido: 10 15 20 30 35 40 45 50 55
  [OK]    EB-N1-B sigue vacio | esperado: true | obtenido: true
  [OK]    Matricula 55 ya existe en el destino (contexto auxiliar) | esperado: false | obtenido: false
  [OK]    El 55 sigue en el origen y el destino no se duplica
  [OK]    Traslado valido de 20 (contexto auxiliar) | esperado: true | obtenido: true
  [OK]    El 20 ya no esta en el origen | esperado: null | obtenido: null
  [OK]    El 20 llega al destino con los mismos datos | esperado: [20] Estudiante 20 | CI 0000000020 | promedio 8,00 | obtenido: [20] Estudiante 20 | CI 0000000020 | promedio 8,00

==================================================
MISION 4 - Modificacion segura y casos de eliminacion del ABB
==================================================
  [OK]    Modificar promedio de 35 a 9.80 | esperado: true | obtenido: true
  [OK]    Promedio de 35 actualizado | esperado: 9.8 | obtenido: 9.8
  [OK]    Actualizar 35 con cedula invalida '123' es rechazado
  [OK]    Actualizar 35 con promedio 15 es rechazado
  [OK]    El registro valido de 35 no cambio | esperado: [35] Estudiante 35 | CI 0000000035 | promedio 9,80 | obtenido: [35] Estudiante 35 | CI 0000000035 | promedio 9,80
  [OK]    Modificar una matricula inexistente (99) | esperado: false | obtenido: false

Traslado de 10 (nodo hoja en el origen)
  Inorden origen antes:   10 15 20 30 35 40 45 50 55
  Inorden origen despues: 15 20 30 35 40 45 50 55
  Inorden destino:        10
  [OK]    Traslado de 10 | esperado: true | obtenido: true
  [OK]    Caso de eliminacion de 10 | esperado: nodo hoja | obtenido: nodo hoja
  [OK]    El origen conserva el orden | esperado: true | obtenido: true
```

Figura 10. *Fin de la Misión 2, Misión 3 completa y comienzo de la Misión 4: modificación de la matrícula 35 y traslado de 10 (nodo hoja).*

6.4. Misión 4: modificación segura y casos de eliminación del ABB (Luis Silva)

Se modificó el promedio de la matrícula 35 a 9.80. Después se intentó actualizarla con una cédula inválida y con promedio 15: ambos intentos se rechazaron y el registro válido no cambió, porque `modificar` valida con un objeto temporal antes de aplicar los cambios.

Luego se trasladaron las matrículas 10, 40, 45 y 30 de EB-N1-A a EB-N1-B. En el ABB de origen, esas eliminaciones cubren los cuatro casos pedidos.

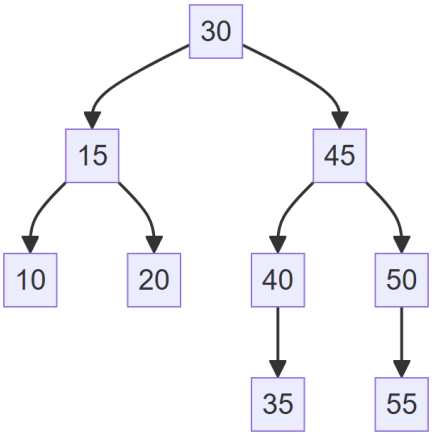


Figura 11. *ABB inicial de EB-N1-A (inserción 30, 15, 45, 10, 20, 40, 50, 35, 55).*

Tabla 6. *Traslados y casos de eliminación en EB-N1-A.*

Matrícula	Caso	Inorden del origen antes	Inorden del origen después
10	Nodo hoja	10 15 20 30 35 40 45 50 55	15 20 30 35 40 45 50 55
40	Nodo con un hijo	15 20 30 35 40 45 50 55	15 20 30 35 45 50 55
45	Nodo con dos hijos	15 20 30 35 45 50 55	15 20 30 35 50 55
30	Raíz	15 20 30 35 50 55	15 20 35 50 55

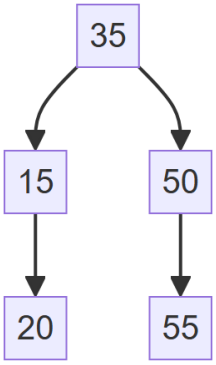


Figura 12. *ABB final de EB-N1-A: 35 como raíz, con 15, 20, 50 y 55.*

```
Traslado de 40 (nodo con un hijo en el origen)
  Inorden origen antes:   15 20 30 35 40 45 50 55
  Inorden origen despues: 15 20 30 35 45 50 55
  Inorden destino:        10 40
  [OK]    Traslado de 40 | esperado: true | obtenido: true
  [OK]    Caso de eliminacion de 40 | esperado: nodo con un hijo | obtenido: nodo con un hijo
  [OK]    El origen conserva el orden | esperado: true | obtenido: true

Traslado de 45 (nodo con dos hijos en el origen)
  Inorden origen antes:   15 20 30 35 45 50 55
  Inorden origen despues: 15 20 30 35 50 55
  Inorden destino:        10 40 45
  [OK]    Traslado de 45 | esperado: true | obtenido: true
  [OK]    Caso de eliminacion de 45 | esperado: nodo con dos hijos | obtenido: nodo con dos hijos
  [OK]    El origen conserva el orden | esperado: true | obtenido: true

Traslado de 30 (raiz en el origen)
  Inorden origen antes:   15 20 30 35 50 55
  Inorden origen despues: 15 20 35 50 55
  Inorden destino:        10 30 40 45
  [OK]    Traslado de 30 | esperado: true | obtenido: true
  [OK]    Caso de eliminacion de 30 | esperado: raiz | obtenido: raiz
  [OK]    El origen conserva el orden | esperado: true | obtenido: true

  [OK]    EB-N1-A final | esperado: 15 20 35 50 55 | obtenido: 15 20 35 50 55
  [OK]    EB-N1-B final | esperado: 10 30 40 45 | obtenido: 10 30 40 45
  [OK]    Repetir el traslado de 10 (ya esta en destino) | esperado: false | obtenido: false

==========================================
MISION 5 - Recorridos y estadisticas comparadas
==========================================
Dato                | EB-N1-A          | EB-N1-B
------------------------------------------------------------
Preorden            | 35 15 20 50 55   | 10 40 30 45
Inorden             | 15 20 35 50 55   | 10 30 40 45
Posorden            | 20 15 55 50 35   | 30 45 40 10
Cantidad            | 5                | 4
Altura (aristas)    | 2                | 2
Matricula menor     | 15               | 10
Matricula mayor     | 55               | 45
Promedio general    | 8.36             | 8.00
Aprobados (>= 8.00) | 5                | 4
Aprobados (>= 9.00) | 1                | 0

  [OK]    Inorden EB-N1-A | esperado: 15 20 35 50 55 | obtenido: 15 20 35 50 55
  [OK]    Inorden EB-N1-B | esperado: 10 30 40 45 | obtenido: 10 30 40 45
  [OK]    Cantidad EB-N1-A | esperado: 5 | obtenido: 5
  [OK]    Cantidad EB-N1-B | esperado: 4 | obtenido: 4
  [OK]    Altura EB-N1-A | esperado: 2 | obtenido: 2
  [OK]    Altura EB-N1-B | esperado: 2 | obtenido: 2
  [OK]    Menor y mayor EB-N1-A | esperado: 15-55 | obtenido: 15-55
  [OK]    Menor y mayor EB-N1-B | esperado: 10-45 | obtenido: 10-45
  [OK]    Promedio EB-N1-A | esperado: 8.36 | obtenido: 8.36
  [OK]    Promedio EB-N1-B | esperado: 8.00 | obtenido: 8.00
  [OK]    Aprobados EB-N1-A con 8.00 | esperado: 5 | obtenido: 5
  [OK]    Aprobados EB-N1-B con 8.00 | esperado: 4 | obtenido: 4
  [OK]    Aprobados EB-N1-A con 9.00 (solo el 35) | esperado: 1 | obtenido: 1
  [OK]    Aprobados en un ABB vacio | esperado: 0 | obtenido: 0
  [OK]    Nota minima 11 rechazada
  [OK]    Nota minima NaN rechazada

Comparacion: EB-N1-A queda con 5 estudiantes y promedio 8.36 porque conserva
al 35 (9.80); EB-N1-B recibe 4 estudiantes con 8.00. Ambos tienen altura 2.
Por que el inorden sale ordenado: en un ABB todo el subarbol izquierdo es menor
que el nodo y todo el derecho es mayor. El inorden visita izquierdo -> nodo ->
derecho, asi que cada matricula aparece despues de todas las menores y antes de
todas las mayores. Por eso los traslados no rompen el orden de ninguno de los dos.
```

Figura 13. *Traslados de 40 (un hijo), 45 (dos hijos) y 30 (raíz) con el inorden antes y después, y comienzo de la Misión 5.*

6.5. Misión 5: recorridos y estadísticas comparadas (Darwin Tisalema)

`contarAprobados(double notaMinima)` recorre recursivamente todo el ABB: un nodo nulo aporta 0, y cada nodo suma 1 si su promedio es mayor o igual a la nota mínima, más lo que aportan sus dos subárboles. Recorre todo el árbol porque el ABB está ordenado por matrícula y no por promedio.

Tabla 7. *Comparación final de los paralelos.*

Dato	EB-N1-A	EB-N1-B
Inorden	15 20 35 50 55	10 30 40 45
Cantidad	5	4
Altura (aristas)	2	2
Matrícula menor / mayor	15 / 55	10 / 45
Promedio general	8.36	8.00
Aprobados (≥ 8.00)	5	4

El promedio de EB-N1-A es (8 + 8 + 9.80 + 8 + 8) / 5 = 8.36, porque conserva a la matrícula 35. El **inorden conserva el orden** de las matrículas porque en un ABB todo el subárbol izquierdo es menor que el nodo y todo el derecho es mayor; al visitar izquierdo → nodo → derecho, cada matrícula sale después de las menores y antes de las mayores.

```
================================================
MISION 5 - Recorridos y estadisticas comparadas
================================================
Dato                  | EB-N1-A          | EB-N1-B
------------------------------------------------------------
Preorden              | 35 15 20 50 55   | 10 40 30 45
Inorden               | 15 20 35 50 55   | 10 30 40 45
Posorden              | 20 15 55 50 35   | 30 45 40 10
Cantidad              | 5                | 4
Altura (aristas)      | 2                | 2
Matricula menor       | 15               | 10
Matricula mayor       | 55               | 45
Promedio general      | 8.36             | 8.00
Aprobados (>= 8.00)   | 5                | 4
Aprobados (>= 9.00)   | 1                | 0

  [OK]    Inorden EB-N1-A | esperado: 15 20 35 50 55 | obtenido: 15 20 35 50 55
  [OK]    Inorden EB-N1-B | esperado: 10 30 40 45 | obtenido: 10 30 40 45
  [OK]    Cantidad EB-N1-A | esperado: 5 | obtenido: 5
  [OK]    Cantidad EB-N1-B | esperado: 4 | obtenido: 4
  [OK]    Altura EB-N1-A | esperado: 2 | obtenido: 2
  [OK]    Altura EB-N1-B | esperado: 2 | obtenido: 2
  [OK]    Menor y mayor EB-N1-A | esperado: 15-55 | obtenido: 15-55
  [OK]    Menor y mayor EB-N1-B | esperado: 10-45 | obtenido: 10-45
  [OK]    Promedio EB-N1-A | esperado: 8.36 | obtenido: 8.36
  [OK]    Promedio EB-N1-B | esperado: 8.00 | obtenido: 8.00
  [OK]    Aprobados EB-N1-A con 8.00 | esperado: 5 | obtenido: 5
  [OK]    Aprobados EB-N1-B con 8.00 | esperado: 4 | obtenido: 4
  [OK]    Aprobados EB-N1-A con 9.00 (solo el 35) | esperado: 1 | obtenido: 1
  [OK]    Aprobados en un ABB vacio | esperado: 0 | obtenido: 0
  [OK]    Nota minima 11 rechazada
  [OK]    Nota minima NaN rechazada

Comparacion: EB-N1-A queda con 5 estudiantes y promedio 8.36 porque conserva
al 35 (9.80); EB-N1-B recibe 4 estudiantes con 8.00. Ambos tienen altura 2.
Por que el inorden sale ordenado: en un ABB todo el subarbol izquierdo es menor
que el nodo y todo el derecho es mayor. El inorden visita izquierdo -> nodo ->
derecho, asi que cada matricula aparece despues de todas las menores y antes de
todas las mayores. Por eso los traslados no rompen el orden de ninguno de los dos.

================================================
MISION 6 - Integracion y reinicio
================================================
  [OK]    Ejecutar el reto dos veces deja exactamente el mismo estado
Despues de reiniciar (opcion 7):
  [OK]    Jerarquia igual a la inicial (sin SW ni sus subunidades)
  [OK]    Unidad SW eliminada por el reinicio | esperado: null | obtenido: null
  [OK]    Unidad SW-N3-A eliminada por el reinicio | esperado: null | obtenido: null
  [OK]    Inorden de EB-N1-A con las 9 matriculas | esperado: 10 15 20 30 35 40 45 50 55 | obtenido: 10 15 20 30 35 40 45 50 55
  [OK]    EB-N1-B vacio | esperado: true | obtenido: true
  [OK]    Matricula 35 vuelve a promedio 8.00 | esperado: 8.0 | obtenido: 8.0

================================================
RESUMEN
================================================
Resultado: 97/97 verificaciones OK
El estado final del reto queda cargado. Use la opcion 7 para volver a los datos iniciales.
```

Figura 14. *Misión 5: comparación de EB-N1-A y EB-N1-B, seguida de la Misión 6 y el resumen 97/97.*

6.6. Misión 6: integración y reinicio (Damian Cunalata)

La opción 8 del menú, «Ejecutar reto integrador», corre las misiones en secuencia y muestra cada comprobación con el valor esperado y el obtenido. Siempre empieza reiniciando los datos, así que ejecutarla varias veces da el mismo resultado sin duplicar unidades ni estudiantes. La opción 7 crea un árbol nuevo: las nueve matrículas vuelven a EB-N1-A, EB-N1-B queda vacío y las unidades del reto desaparecen.

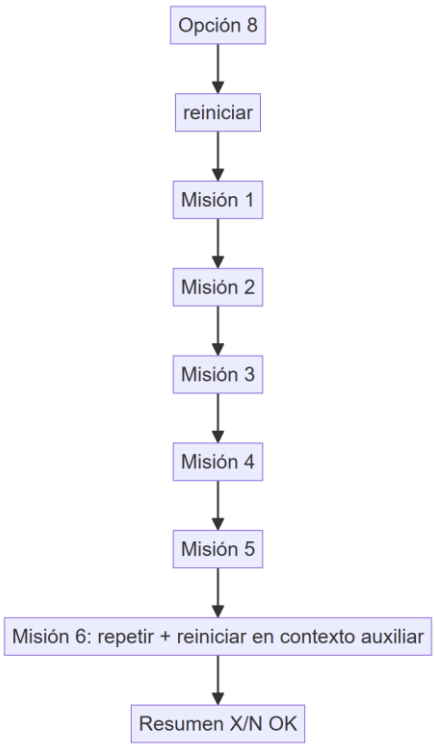


Figura 15. *Secuencia de la opción 8.*

```
Comparacion: EB-N1-A queda con 5 estudiantes y promedio 8.36 porque conserva
al 35 (9.80); EB-N1-B recibe 4 estudiantes con 8.00. Ambos tienen altura 2.
Por que el inorden sale ordenado: en un ABB todo el subarbol izquierdo es menor
que el nodo y todo el derecho es mayor. El inorden visita izquierdo -> nodo ->
derecho, asi que cada matricula aparece despues de todas las menores y antes de
todas las mayores. Por eso los traslados no rompen el orden de ninguno de los dos.

================================================
MISION 6 - Integracion y reinicio
================================================
  [OK]    Ejecutar el reto dos veces deja exactamente el mismo estado
Despues de reiniciar (opcion 7):
  [OK]    Jerarquia igual a la inicial (sin SW ni sus subunidades)
  [OK]    Unidad SW eliminada por el reinicio | esperado: null | obtenido: null
  [OK]    Unidad SW-N3-A eliminada por el reinicio | esperado: null | obtenido: null
  [OK]    Inorden de EB-N1-A con las 9 matriculas | esperado: 10 15 20 30 35 40 45 50 55 | obtenido: 10 15 20 30 35 40 45 50 55
  [OK]    EB-N1-B vacio | esperado: true | obtenido: true
  [OK]    Matricula 35 vuelve a promedio 8.00 | esperado: 8.0 | obtenido: 8.0

================================================
RESUMEN
================================================
Resultado: 97/97 verificaciones OK
El estado final del reto queda cargado. Use la opcion 7 para volver a los datos iniciales.


====================================
    SISTEMA JERARQUICO ACADEMICO UTA
      APE3 - RETO INTEGRADOR - GRUPO 2
====================================
1. Jerarquia academica
2. Insercion y busqueda de estudiantes
3. Recorridos y estadisticas
4. Modificacion y eliminacion de estudiantes
5. Unidades y transformacion binaria
6. Estado general del sistema
7. Reiniciar datos de ejemplo
8. Ejecutar reto integrador
0. Salir
====================================
Seleccione una opcion:
```

Figura 16. *Final de la opción 8: Misión 6, resumen de 97/97 verificaciones y regreso al menú.*

```
====================================
    SISTEMA JERARQUICO ACADEMICO UTA
      APE3 - RETO INTEGRADOR - GRUPO 2
====================================
1. Jerarquia academica
2. Insercion y busqueda de estudiantes
3. Recorridos y estadisticas
4. Modificacion y eliminacion de estudiantes
5. Unidades y transformacion binaria
6. Estado general del sistema
7. Reiniciar datos de ejemplo
8. Ejecutar reto integrador
0. Salir
====================================
Seleccione una opcion: 7

Datos reiniciados correctamente.

====================================
    SISTEMA JERARQUICO ACADEMICO UTA
      APE3 - RETO INTEGRADOR - GRUPO 2
====================================
1. Jerarquia academica
2. Insercion y busqueda de estudiantes
3. Recorridos y estadisticas
4. Modificacion y eliminacion de estudiantes
5. Unidades y transformacion binaria
6. Estado general del sistema
7. Reiniciar datos de ejemplo
8. Ejecutar reto integrador
0. Salir
====================================
Seleccione una opcion: 6

===== ESTADO GENERAL SJA-UTA =====

--- Arbol organizacional ---
Altura: 4
Hojas: 2

--- Paralelo EB-N1-A ---
Nivel del paralelo: 4
Cantidad: 9
Altura: 3
Menor matricula: 10
Mayor matricula: 55
Promedio general: 8,00

--- Paralelo EB-N1-B ---
Nivel del paralelo: 4
Cantidad: 0
Altura: -1
No hay estudiantes registrados.

Diagnostico finalizado.
```

Figura 17. *Opción 7 (reinicio) y opción 6: 2 hojas, EB-N1-A con 9 estudiantes y EB-N1-B vacío.*

7. Compilación y verificaciones obligatorias

El proyecto se compiló en la terminal de VS Code con JDK 21: `javac` no muestra errores y el código de salida es 0 (figura 18).

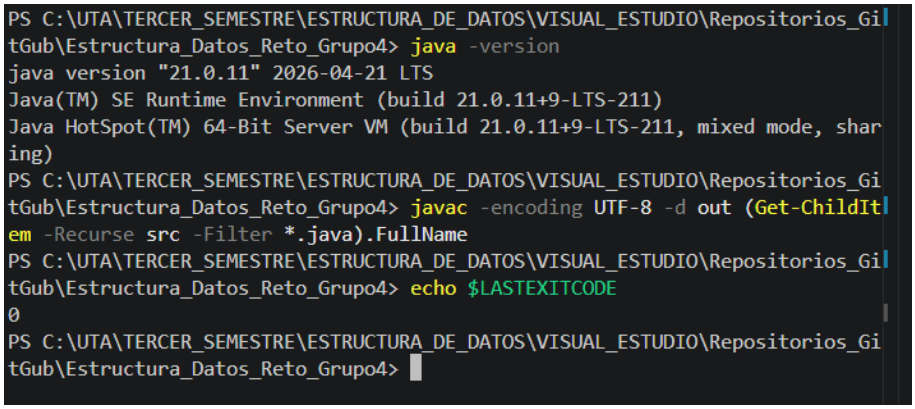


Figura 18. *Compilación del proyecto con Java 21 en la terminal de VS Code.*

La tabla 8 compara cada verificación obligatoria del enunciado con el resultado de la ejecución real, y la tabla 9 resume las verificaciones por misión.

Tabla 8. *Verificaciones obligatorias.*

Verificación del enunciado	Esperado	Obtenido
Ruta de EB-N1-A	UTA -> FCHE -> EB -> EB-N1 -> EB-N1-A	Igual
Eliminar EB-N1-A con estudiantes	Rechazado (false)	false
Eliminar SW-N3-A como hoja vacía	Permitido (true)	true
EB-N1-A tras los traslados	15, 20, 35, 50, 55	15 20 35 50 55
EB-N1-B tras los traslados	10, 30, 40, 45	10 30 40 45
Promedio de EB-N1-A / EB-N1-B	8.36 / 8.00	8.36 / 8.00
Reinicio con la opción 7	Datos originales recuperados	Cumple

Tabla 9. *Verificaciones por misión en la ejecución real.*

Misión	Verificaciones correctas
1	15
2	15
3	23
4	21
5	16
6	7
Total	**97 / 97**

8. Evidencias de commits por integrante

A continuación se muestra el commit más representativo de cada integrante, tal como aparece en GitHub.

8.1. Damian Cunalata

Commit `30cc365`: integra el árbol organizacional, la ruta jerárquica y la opción 8 del reto. Agrega ArbolOrganizacional, RutaJerarquica, Contexto, Main, RetoIntegrador, RetoMision1 y la documentación de las Misiones 1 y 6 (8 archivos).

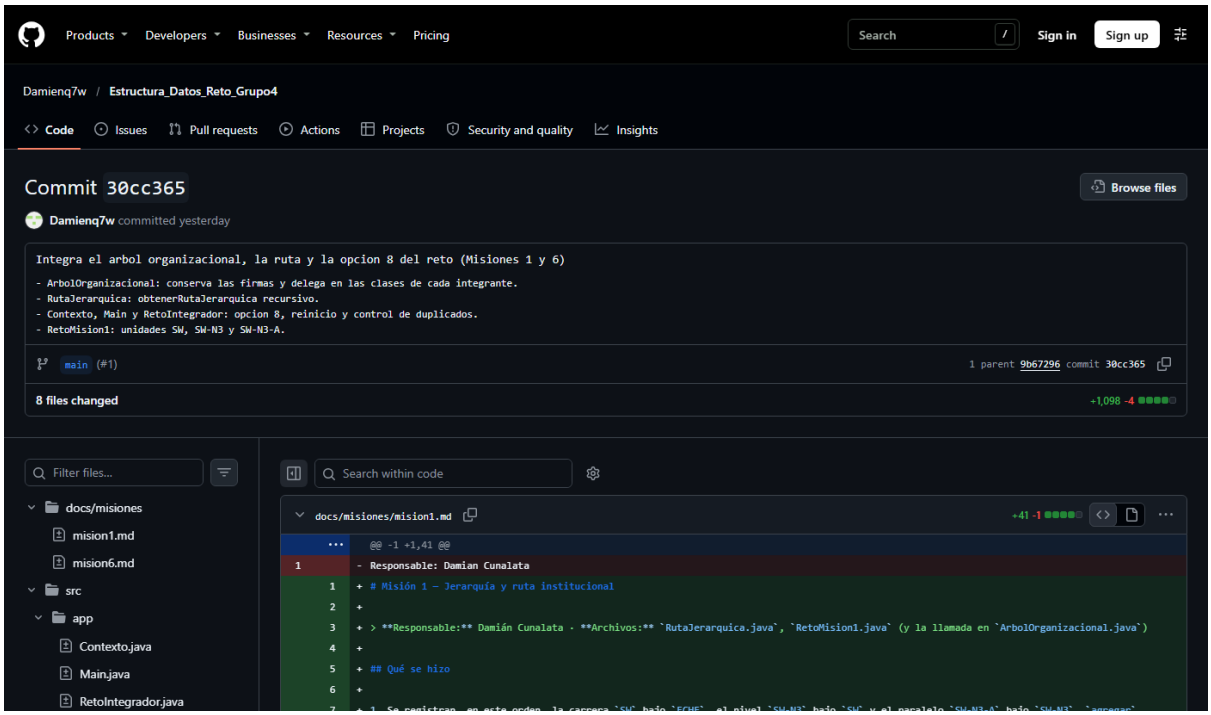


Figura 19. *Commit 30cc365 de Damian Cunalata en GitHub.*

8.2. Mónica Tacuri

Commit `ce9b1c8`: implementa la eliminación protegida y la transformación binaria: NodoUnidad, NodoBinario, EliminadorUnidades, TransformadorBinario y la documentación de la Misión 2 (5 archivos).

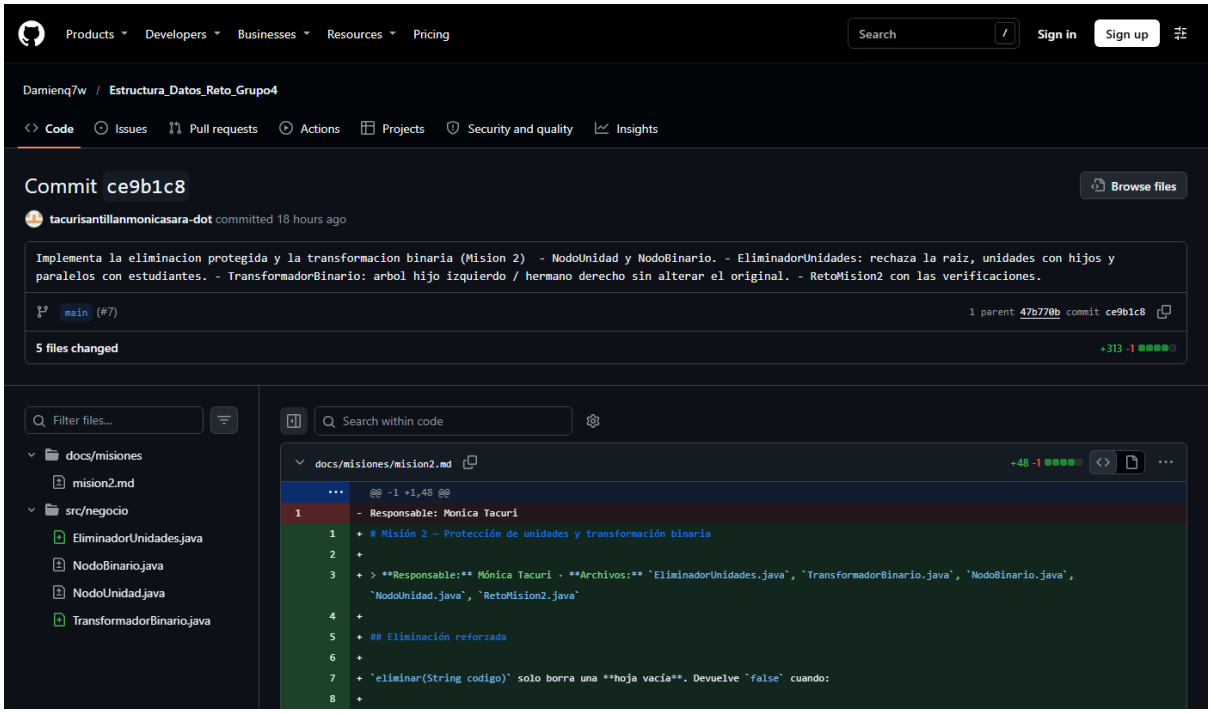


Figura 20. *Commit ce9b1c8 de Mónica Tacuri en GitHub.*

8.3. Kenneth Chalco

Commit `f41ef0b`: implementa el modelo y el traslado seguro: Estudiante, UnidadAcademica, TipoUnidad, TrasladoEstudiantes, RetoMision3 y la documentación de la Misión 3 (6 archivos).

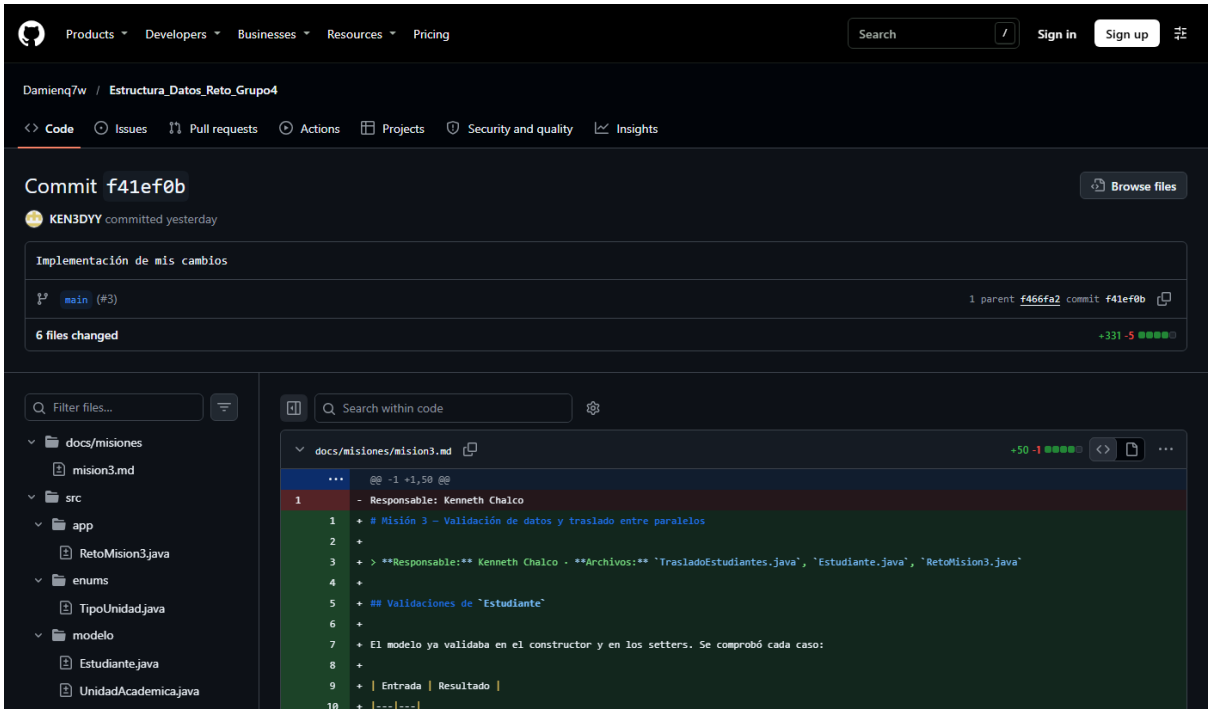


Figura 21. *Commit f41ef0b de Kenneth Chalco en GitHub.*

8.4. Luis Silva

Commit `559e865`: agrega el reporte de verificaciones y la modificación segura del ABB: NodoEstudiante, Reporte, RetoMision4 y la documentación de la Misión 4 (4 archivos).

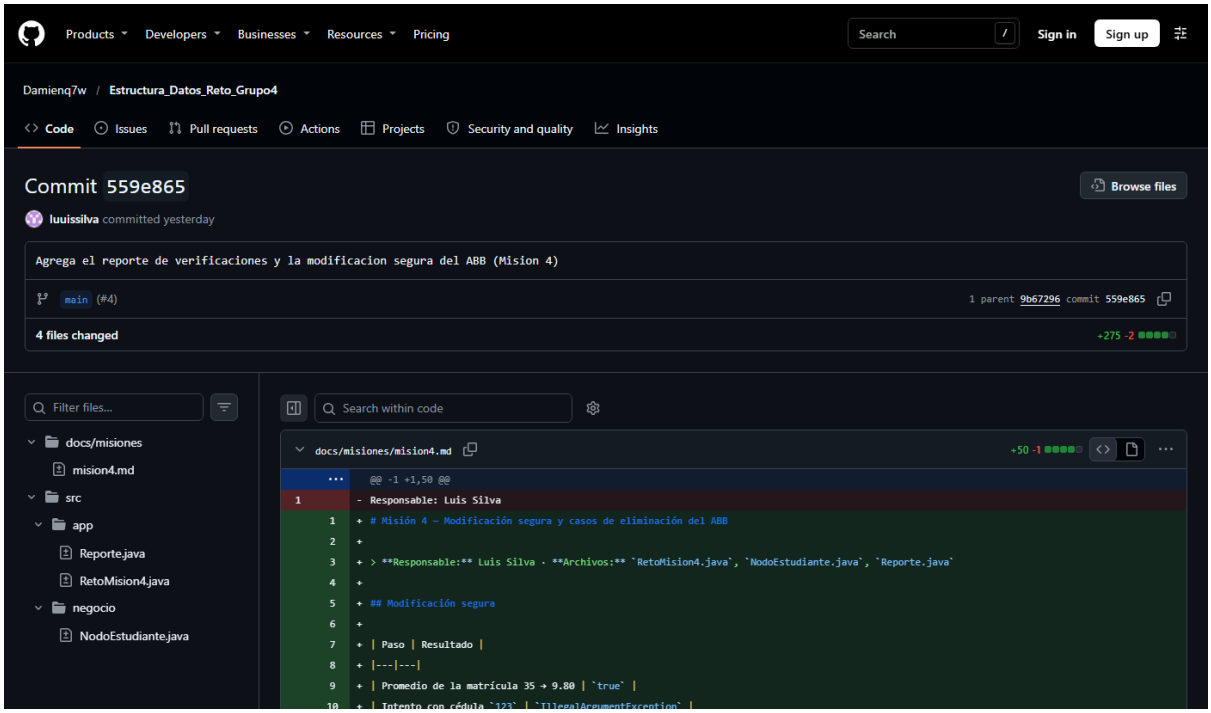


Figura 22. *Commit 559e865 de Luis Silva en GitHub.*

8.5. Darwin Tisalema

Commit `7f9fa65`: implementa las operaciones recursivas del ABB de estudiantes en ArbolEstudiantes, incluida contarAprobados. En commits aparte subió TipoRecorrido y RetoMision5.

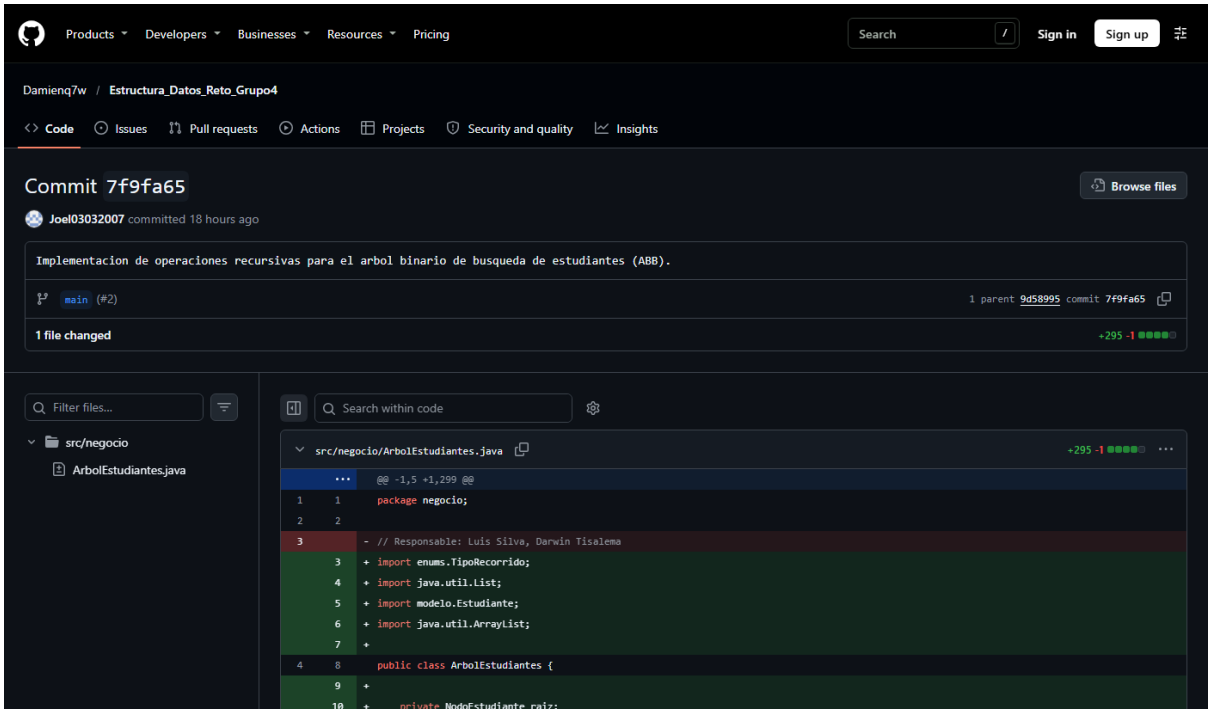


Figura 23. *Commit 7f9fa65 de Darwin Tisalema en GitHub.*

8.6. Josue Jampier Camacho

Commit `8fe1392`: documenta los resultados y las evidencias de ejecución del reto en docs/evidencias.md. También redactó el README y las decisiones de implementación.

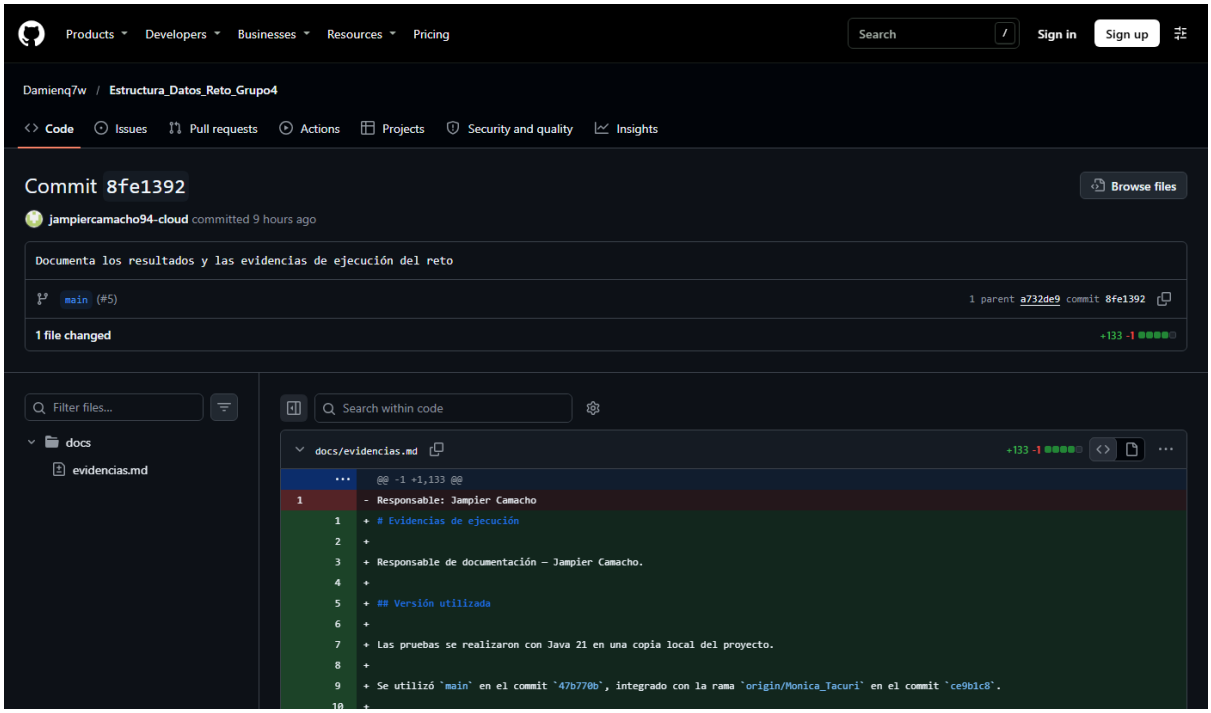


Figura 24. *Commit 8fe1392 de Josue Jampier Camacho en GitHub.*

9. Decisiones de implementación

Tabla 10. *Principales decisiones de implementación.*

Decisión	Motivo
ArbolOrganizacional delega en clases auxiliares	Conserva las firmas del enunciado y cada clase tiene un único responsable
Proteger la raíz UTA	Sin raíz el sistema queda inutilizable hasta reiniciar
Crear nodos nuevos para la transformación binaria	Es una transformación real y garantiza que el árbol general no cambie
Insertar en el destino antes de eliminar del origen	Si un paso falla, el estudiante no se pierde
Trasladar una copia del estudiante	Los dos ABB nunca comparten el mismo objeto
Validar con un objeto temporal antes de modificar	Un dato inválido no deja el registro a medio cambiar
La opción 8 reinicia antes de ejecutar	Repetir el reto no duplica unidades ni estudiantes
Mostrar esperado y obtenido en cada verificación	El enunciado pide resultados de una ejecución real

10. Conclusiones

- Se cumplieron las seis misiones del reto y todas las verificaciones obligatorias; la ejecución real terminó con 97 de 97 verificaciones correctas.
- La recursión resultó natural para ambos árboles: la ruta, la eliminación y la transformación recorren el árbol general, y los recorridos, la altura y el conteo de aprobados recorren el ABB.
- Insertar primero en el destino y deshacer si falla la eliminación evita perder estudiantes durante un traslado, y el inorden confirmó que el ABB conserva el orden después de cada eliminación.
- Asignar un solo responsable por clase, con ramas y pull requests, dejó en GitHub un registro claro del aporte de cada integrante.

11. Recomendaciones

- Equilibrar el ABB (por ejemplo, con un árbol AVL) si se insertan matrículas ya ordenadas, porque el árbol se degenera en una lista y las búsquedas pasan de O(log n) a O(n).
- Compilar y ejecutar `java -cp out app.RetoIntegrador` antes de cada pull request, para detectar a tiempo archivos incompletos.
- Reemplazar la regla de jerarquía basada en el orden del enumerado por una relación explícita entre tipos, para que reordenar las constantes no la rompa.

12. Anexos

La salida completa de la ejecución del reto está en `docs/salida_reto.txt` del repositorio. Las siguientes capturas muestran el historial de commits de los 6 integrantes y un pull request fusionado.

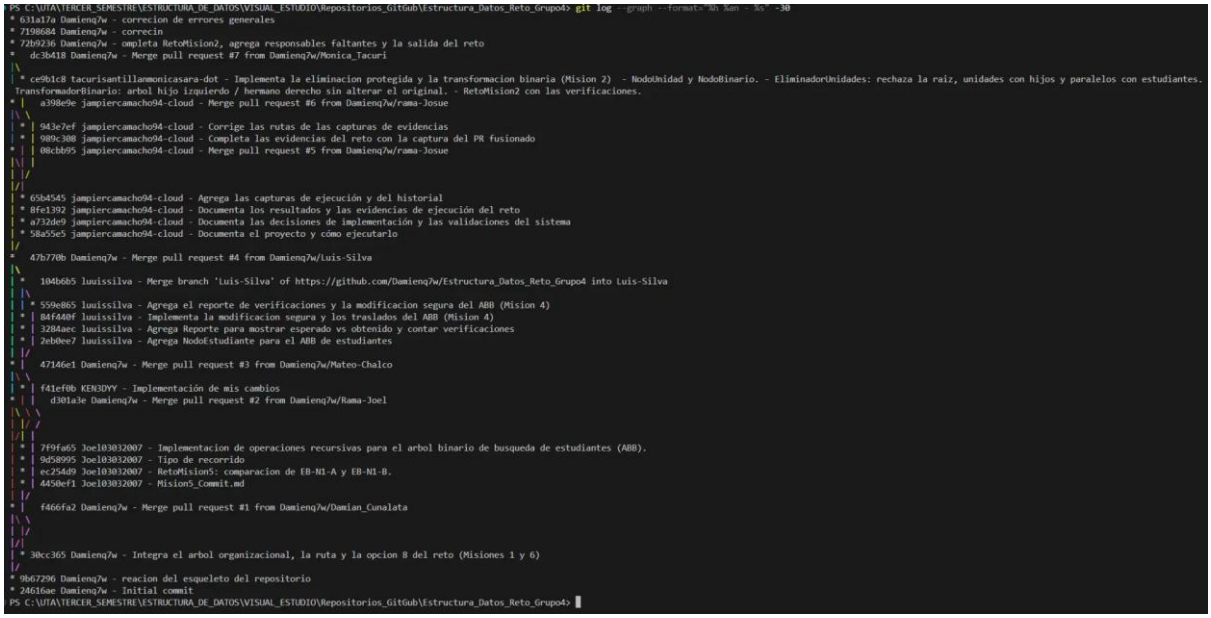


Figura 25. *Historial de commits con git log --graph: aportes y pull requests de los 6 integrantes.*

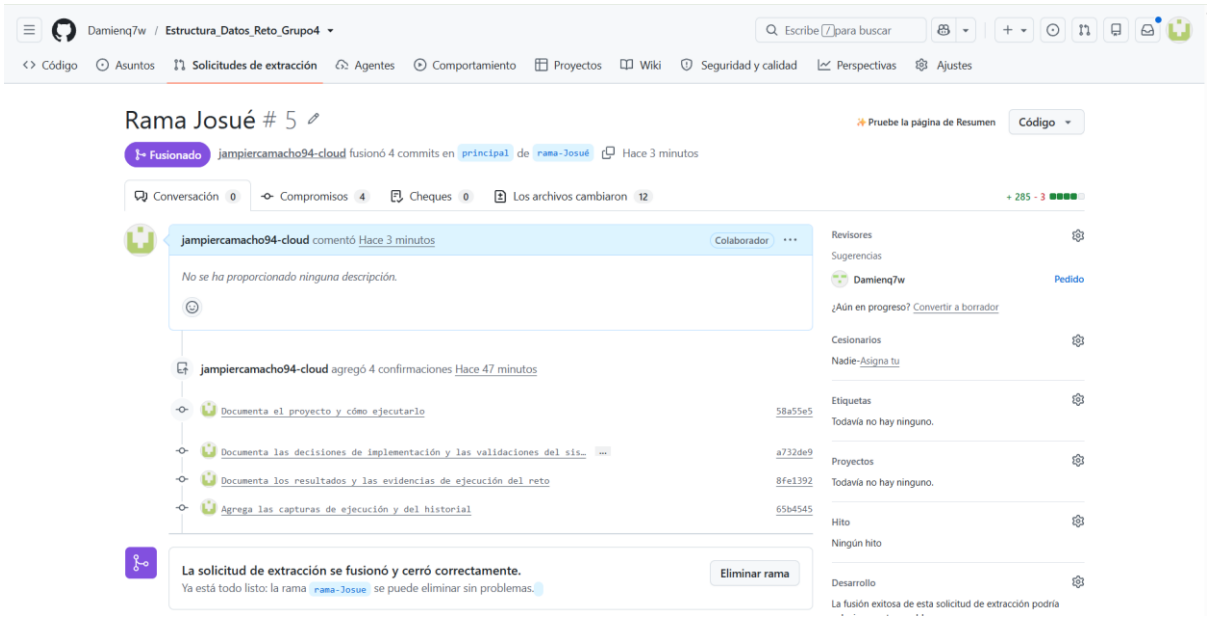


Figura 26. *Pull request fusionado en el historial.*